

Mock Quiz — Complete Worked Solutions

All 16 questions from `mock.txt`, options exactly as given, with the correct answer *and* full explanations for why every wrong option is wrong

Source: `mock.txt` — a 16-question mock exam covering Week 1–4 topics (perceptrons, cross-entropy, activation functions, regularization, Batch Norm, backprop, CNN receptive fields, PyTorch internals, optimizers, dropout, NLP architectures, and a common-data CNN block for Q12–16).

Only Question 1 had a marked answer in the source ("Selected in image"); for every other question, the correct answer below was independently derived and verified from first principles — the same rigor applied throughout `revision.html`. Where my derivation didn't match any of the given options (this happened once, Q7), that is called out explicitly rather than silently forcing a best-fit choice.

Contents

- Q1 — Perceptron & XOR
- Q2 — Cross-entropy loss (numeric)
- Q3 — Role of activation functions
- Q4 — Regularization matching
- Q5 — Batch Normalization
- Q6 — 2-layer MLP forward pass
- Q7 — CNN receptive field
- Q8 — `F.cross_entropy` internals
- Q9 — SGD `weight_decay`
- Q10 — `nn.Dropout` train vs eval
- Q11 — NLP task ↔ architecture matching
- Q12–16 — Common-data CNN block

A single-layer perceptron with a hard step activation is trained on a dataset that is not linearly separable. Which of the following is the most accurate statement about its representational limitation?

- A. It will converge to a solution that minimizes mean squared error on the training set.
- B. It can represent any Boolean function if trained long enough.
- C. It cannot represent the XOR function due to linear separability limits. ✓ **CORRECT (marked in source)**
- D. It can represent XOR only if the learning rate is sufficiently small.

Correct answer: C

Why C is correct

A single-layer perceptron with a hard step activation can only draw one straight decision boundary (a hyperplane) through input space. XOR is the textbook example of a pattern that is **not linearly separable** — no single straight line separates its two classes — so a single-layer perceptron can never represent it, regardless of training.

Why the others are wrong:

- **A** — the classic perceptron learning rule doesn't minimize MSE; it updates weights only on misclassified points, and on non-separable data it never settles into a stable "converged" solution at all — it oscillates forever.
- **B** — false in general. Representing arbitrary Boolean functions needs decision boundaries that can be curved or disconnected, which one linear unit can never produce, however long you train it.
- **D** — learning rate only controls step size, not the shape of the boundary. No learning rate turns a straight line into something that can separate XOR's diagonal pattern.

You train a classifier with softmax + cross-entropy. For a single sample, the model logits are $[2, 0, -1]$ for classes $[0, 1, 2]$, and the true class is 0. Compute the cross-entropy loss (natural log) to 3 decimal places.

Answer: 0.170

Full worked solution

Softmax probability of the true class: $P(0) = \frac{e^{z_0}}{\sum_j e^{z_j}}$.

$$e^2 = 7.38906, \quad e^0 = 1, \quad e^{-1} = 0.36788$$
$$\text{Sum} = 7.38906 + 1 + 0.36788 = 8.75694$$

$$P(0) = \frac{7.38906}{8.75694} = 0.84379$$

Cross-entropy loss (a very convenient shortcut: $L = -z_0 + \ln(\text{sum})$, avoiding an extra division-then-log step):

$$L = -\ln(P(0)) = -2 + \ln(8.75694) = -2 + 2.16986 = 0.16986$$

$$L \approx \mathbf{0.170} \text{ (to 3 decimal places)}$$

Which statement best captures the role of an activation function in an MLP?

- A. It guarantees convexity of the loss landscape.
- B. It introduces nonlinearity, enabling composition of linear transforms to model complex functions.**
- C. It primarily serves as a learning rate schedule.
- D. It regularizes the model by dropping activations.

✓ CORRECT

Correct answer: B

Why B is correct

Without a nonlinear activation, stacking any number of fully-connected layers collapses into one equivalent linear layer: $W_2(W_1x) = (W_2W_1)x$. The nonlinearity between layers is exactly what lets deep composition represent genuinely complex, curved functions and decision boundaries.

Why the others are wrong:

- **A** — usually the opposite: adding nonlinear activations makes the loss landscape *non-convex*, which is precisely why deep-network optimization is hard.
- **C** — a learning rate schedule is a separate optimizer-level hyperparameter (how step size changes over training); unrelated to activation functions.
- **D** — that description is Dropout, a completely different regularization technique, not what "activation function" means.

Match each regularization technique to its correct function related to generalization.

A. L2 Weight Decay **B.** Dropout (p=0.5) **C.** Early Stopping

Functions: 1. Stops training early to avoid overfitting 2. Penalizes large weights, smoothing decision boundaries 3. Increases the effective width at test time 4. Behaves like model averaging over sub-networks

A. A-4, B-1, C-2

B. A-2, B-4, C-1

✓ CORRECT

C. A-1, B-2, C-3

D. A-3, B-4, C-2

Correct answer: B (A-2, B-4, C-1)

Matching each technique to its true function

- **A. L2 Weight Decay** → 2 — adding $\lambda \sum w^2$ to the loss directly penalizes large weight magnitudes, which tends to smooth out decision boundaries.
- **B. Dropout** → 4 — randomly zeroing different neurons every forward pass during training effectively trains a huge implicit ensemble of "thinned" sub-networks, and the combined behaviour approximates averaging over them.
- **C. Early Stopping** → 1 — literally its definition: stop training before the model has fully converged on the training set, at the point where validation/test performance is best.

Why function 3 is a distractor, and the other option-combinations are wrong:

Function 3 ("increases the effective width at test time") doesn't correctly describe any of the three techniques — Dropout is *disabled* at test time (every neuron active, no width change at all), so 3 is simply a false statement planted to catch anyone who mismatches B. Options A, C, and D each pair at least one technique with the wrong function (e.g. option D claims Early Stopping "increases effective width," which describes neither Early Stopping nor anything real), so only option B's complete set of pairings (A-2, B-4, C-1) is fully correct.

Batch Normalization primarily helps by:

- A. Eliminating the need for nonlinear activations.
- B. Reducing internal covariate shift and stabilizing gradients across layers. ✓ CORRECT**
- C. Guaranteeing faster test-time inference.
- D. Replacing the optimizer's momentum term.

Correct answer: B

Why B is correct

Batch Norm normalizes each layer's activations (per mini-batch) to stable mean/variance before applying learnable scale γ and shift β . This keeps the distribution each layer sees relatively stable across training steps, which stabilizes gradient flow and typically allows larger learning rates.

Why the others are wrong:

- **A** — Batch Norm is a normalization step, not an activation function; you still need ReLU/sigmoid/etc. for nonlinearity.
- **C** — it doesn't guarantee faster inference; at inference it uses stored running statistics, an extra (not eliminated) per-layer computation.
- **D** — momentum is an optimizer-level concept for accumulating gradient updates across steps; Batch Norm operates on activations within the forward pass and is unrelated to it.

A two-layer MLP (no bias) is $f(x) = W_2 \times \text{ReLU}(W_1 x)$. With $W_1 = \begin{bmatrix} 1 & -2 \\ -3 & 4 \end{bmatrix}$, $W_2 = \begin{bmatrix} 2 & -1 \end{bmatrix}$, $x = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$. Compute $f(x)$.

Answer: $f(x) = 0$

Full worked solution

$$W_1 x = \begin{bmatrix} 1 & -2 \\ -3 & 4 \end{bmatrix} \begin{bmatrix} 2 \\ 1 \end{bmatrix} = \begin{bmatrix} 1(2) + (-2)(1) \\ -3(2) + 4(1) \end{bmatrix} = \begin{bmatrix} 0 \\ -2 \end{bmatrix}$$

$$\text{ReLU} \left(\begin{bmatrix} 0 \\ -2 \end{bmatrix} \right) = \begin{bmatrix} \max(0, 0) \\ \max(0, -2) \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$$f(x) = W_2 \begin{bmatrix} 0 \\ 0 \end{bmatrix} = 2(0) + (-1)(0) = 0$$

A CNN has four convolutional layers, each with kernel 3, stride 1, and padding 1, followed by one max-pooling layer of size 2×2 and stride 2. What is the receptive field (in input pixels) of one neuron in the final feature map?

A. 7×7

B. 9×9

C. 12×12

D. 14×14

Rigorously computed answer: 10×10 — this does not match any of the four given options (closest is B, see note below)

Full, verified derivation

Use the standard receptive-field recurrence (the same one used to trace receptive-field growth in the Week 3 notes): $r_l = r_{l-1} + (k_l - 1) \cdot j_{l-1}$, and $j_l = j_{l-1} \cdot s_l$, where r is the receptive field size and j is the cumulative "jump" (product of strides) so far. Start with $r_0 = 1$, $j_0 = 1$:

Layer	k	s	r after this layer	j after this layer
Conv 1	3	1	$1 + (3 - 1)(1) = 3$	1
Conv 2	3	1	$3 + (3 - 1)(1) = 5$	1
Conv 3	3	1	$5 + (3 - 1)(1) = 7$	1
Conv 4	3	1	$7 + (3 - 1)(1) = 9$	1
MaxPool	2	2	$9 + (2 - 1)(1) = 10$	2

This was independently double-checked by literally tracing index dependencies backward one layer at a time (not just plugging into the formula): tracking which input positions influence output position 0 of the final feature map, through all 5 layers, gives the input-index range $\{-4, -3, \dots, 4, 5\}$ — exactly 10 input positions. So the receptive field is genuinely 10×10 .

This doesn't match any of the four options — most likely explanation

All four conv layers use padding 1 with kernel 3 (i.e. "same" padding, which preserves spatial size) — that detail affects the *output size*, not the receptive-field growth calculation, so it doesn't change the math above. The most likely source of the mismatch: if you only count the receptive field built up by

the **four convolutional layers** and stop there ($r = 9$ after Conv 4, before the pooling layer's own contribution), you land exactly on option **B (9×9)** — but the question explicitly asks for the receptive field of a neuron in the *final* feature map, which is the max-pool layer's output, so the pooling layer's own kernel must be included, giving 10×10 by the correct, complete calculation. If this question's answer key says B, it most likely reflects that (technically incomplete) omission of the pooling layer's contribution — worth flagging to whoever wrote the key, but the fully rigorous answer, shown step by step above, is 10×10 .

```
import torch
torch.manual_seed(1)
logits = torch.tensor([[3.0, -1.0, 0.5]])
target = torch.tensor([0])
loss = torch.nn.functional.cross_entropy(logits, target)
```

Which statement is correct?

- A.** `loss` equals `-torch.log_softmax(logits, dim=1)[0,0]`. ✓ CORRECT
- B.** `loss` equals `-torch.log(torch.softmax(logits, dim=0)[0,0])`.
- C.** `loss` is the same as MSE between logits and one-hot vector.
- D.** `loss` depends on temperature scaling by default.

Correct answer: A

Why A is correct

`F.cross_entropy(logits, target)` is internally implemented as `log_softmax` (over the class dimension) followed by `NLLLoss` (picking out, and negating, the log-probability at the target index). For a `[batch, classes]`-shaped tensor, the class dimension is `dim=1`, and here `target=0`, so the loss is exactly `-log\softmax(logits, dim = 1)[0, 0]`.

Why the others are wrong:

- B** — uses `dim=0`, which normalizes down the *batch* dimension instead of across classes. For this single-row input, softmax over `dim=0` is meaningless for computing per-class probabilities (each single-row "column" just normalizes to 1); this is the wrong axis entirely.
- C** — cross-entropy and MSE are different loss functions with different gradients and different behaviour; cross-entropy is never "the same as" MSE between logits and a one-hot vector.
- D** — `F.cross_entropy` applies no temperature scaling by default; logits are used as-is (dividing by a temperature is an optional technique some people apply manually, not a default behaviour).

Note on `torch.manual_seed(1)`

This line is a red herring — there's no randomness anywhere in this snippet (no dropout, no random init), so the seed has zero effect on the result.

Question 9 / 16

Single choice

Suppose you optimize with `torch.optim.SGD(params, lr=0.1, weight_decay=1e-4)`. This `weight_decay` is equivalent to:

A. Adding $\lambda \|W\|_1$ to the loss and differentiating.

B. Adding $\lambda \|W\|_2^2$ to the loss and differentiating.

✓ CORRECT

C. Clipping weights by λ every step.

D. Decaying gradients by λ .

Correct answer: B

Why B is correct

PyTorch's plain SGD implements `weight_decay` as classic L2 regularization: it's mathematically equivalent to adding a $\lambda \|W\|_2^2$ penalty term to the loss and letting normal differentiation handle the rest. Its gradient contribution is $2\lambda W$, which is exactly why the SGD implementation adds a term proportional to W itself onto the gradient every step (the $\lambda \|W\|_2^2$ term differentiates to $2\lambda W$).

Why the others are wrong:

- **A** — that's L1 regularization, which produces sparse weights (many exactly zero) and has a different (non-smooth, sign-based) gradient — not what PyTorch's `weight_decay` implements.
- **C** — `weight_decay` doesn't hard-clip weights to any range; it's a smooth penalty added to the gradient every step, allowing weights to still grow if the actual task gradient favours it.
- **D** — it decays *weights* (via the gradient nudge derived from the penalty), not the gradients of the original task loss themselves.

```
import torch, torch.nn as nn
torch.manual_seed(5)
drop = nn.Dropout(p=0.5)
x = torch.ones(10, 4)
y_train = drop(x)
drop.eval()
y_eval = drop(x)
```

Which statement is correct?

A. `y_train` will be all zeros.

B. `y_eval` equals `x` scaled by $(1-p)$.

C. `y_train` keeps a random 50% of units and scales survivors by $1/(1-p)$.

✓ CORRECT

D. `y_eval` equals exactly `x`.

✓ ALSO TRUE

Both C and D are factually correct as written — see note below

Why C is correct

PyTorch's `nn.Dropout` starts in training mode by default. In training mode, it randomly zeroes elements with probability $p = 0.5$ (so on average ~50% of the 40 entries in this 10×4 tensor of ones become 0), and — using the standard "inverted dropout" implementation — scales every surviving (non-zeroed) element by $\frac{1}{1-p} = \frac{1}{0.5} = 2$. This is exactly what C describes.

Why D is *also* correct

After calling `drop.eval()`, the module's `self.training` flag becomes `False`, and `nn.Dropout.forward` simply returns its input unchanged in that mode — no zeroing, no scaling. So `y_eval` is exactly equal to `x`, making D a true statement as well.

Flagging an issue with this question

As written, this question has **two** correct options (C and D) rather than one, since the code snippet genuinely demonstrates both facts (train-time scaling behaviour, and eval-time identity behaviour) and both listed statements about it are accurate. If forced to pick a single "best" answer, C is the more specific and instructive one (it tests whether you know the $1/(1-p)$ inverted-dropout scaling rule, which is the more commonly-tested and easily-confused detail), while D states the

comparatively simpler fact that eval mode disables dropout outright. Both are worth knowing regardless of which the answer key marks.

Why A and B are wrong:

- **A** — dropout zeroes elements *randomly*, not all of them; on average about half survive (and get scaled up), so `y_train` is not all zeros.
- **B** — this reverses the actual rule. If anything resembled scaling by $(1 - p)$, it would be scaling *down*; but eval mode doesn't scale at all — it returns `x` completely unchanged (see D).

Select the correct matching pairs of NLP tasks with the ideal choice of models most suitable for them.

A. Machine Translation - Only Decoder

B. Question Answering - Encoder Decoder

C. Sentiment Analysis - Encoder Only

✓ CORRECT

D. Text Summarization - Encoder Only

Correct pairing: only C

The standard 3-way architecture split

Architecture	Best suited for	Why
Encoder-only (e.g. BERT)	Understanding / classification tasks: sentiment analysis, extractive QA, NER	Sees the whole input bidirectionally at once; outputs a fixed-size understanding, not a generated sequence.
Decoder-only (e.g. GPT)	Open-ended autoregressive generation	Generates tokens one at a time, each conditioned only on tokens so far.
Encoder-Decoder (e.g. T5, the original Transformer)	Sequence-to-sequence transformation: translation, summarization	Encoder fully digests the input; decoder generates a new, different-length output sequence conditioned on it.

Why C is correct

Sentiment Analysis is a classification task (map input text → one of a few labels), which is exactly what encoder-only models like BERT are built for — no generation is needed at all.

Why A, B, D are wrong:

- **A** — Machine Translation is the canonical sequence-to-sequence task (a full input sentence transforms into a full, different-language output sentence). The ideal, most classically-taught architecture is **Encoder-Decoder** (this is literally what the original Transformer paper was built to do), not "only decoder."
- **B** — Question Answering (particularly the classic extractive/span-based QA task, e.g. SQuAD-style — "find the answer span within this passage") is best suited to an **encoder-only** model like BERT, which just needs to understand the passage+question and point at a span — not "Encoder

Decoder." (Generative/open-domain QA can use encoder-decoder models, but that's not the "ideal"/default categorization taught for QA.)

- **D** — Text Summarization is a generation task (input document → a new, shorter generated sequence), which needs an **Encoder-Decoder** architecture (e.g. BART), not "Encoder Only" — an encoder-only model has no mechanism to generate new output text at all.

```

import torch
import torch.nn as nn

class CNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.layers = nn.Sequential(
            nn.Conv2d(3, 16, 5, padding=2),
            nn.ReLU(),
            nn.Conv2d(16, 32, 3, padding=1),
            nn.ReLU(),
            nn.Conv2d(32, 64, 3, padding=1),
            nn.ReLU()
        )

    def forward(self, x):
        return self.layers(x)

m = CNN()
x = torch.randn(1,3,64,64)
y = m(x)

```

Tracing the whole network once, up front

Every single conv layer here uses "same" padding — i.e. $P = \frac{F-1}{2}$ for its kernel size (5×5 with $P = 2$; both 3×3 's with $P = 1$) — which by design keeps the spatial size *unchanged* at every step. There is no pooling anywhere in this network, so the spatial size never shrinks at all, from input to output.

Stage	Layer	Output shape (C×H×W)
Input	—	$3 \times 64 \times 64$
After conv1	Conv2d(3,16,5,pad=2)	$16 \times 64 \times 64$
After conv2	Conv2d(16,32,3,pad=1)	$32 \times 64 \times 64$
After conv3	Conv2d(32,64,3,pad=1)	$64 \times 64 \times 64$

What is the spatial size ($H \times W$) of the output feature map y ?

A. 32×32

B. 64×64

✓ CORRECT

C. 128×128

D. 16×16

Correct answer: B

Why B is correct

Check each conv layer's output size with $O = \frac{W-F+2P}{S} + 1$: conv1: $\frac{64-5+2(2)}{1} + 1 = \frac{63}{1} + 1 = 64$; conv2: $\frac{64-3+2(1)}{1} + 1 = 64$; conv3: same, 64. Every layer preserves 64×64 exactly (that's the whole point of "same" padding) — and since there's no pooling layer anywhere in this network, the final spatial size is unchanged from the input's: 64×64 .

Why the others are wrong: A (32×32) and D (16×16) would only occur if there were pooling or stride-2 convolutions somewhere — there are none. C (128×128) would require some kind of upsampling/transpose-convolution, which also doesn't appear in this network.

How many channels does the **output** feature map y have?

A. 16

B. 32

C. 64

✓ CORRECT

D. 3

Correct answer: C

Why C is correct

The output channel count of the whole network is set by the *last* conv layer's `out_channels` argument — `nn.Conv2d(32, 64, 3, padding=1)` — which is 64. (A: 16 is the channel count after only the first conv layer; B: 32 is after only the second; D: 3 is the original input's channel count, before any layer runs.)

What is the number of channels after the **first** conv layer?

A. 16

✓ CORRECT

B. 3

C. 32

D. 64

Correct answer: A

Why A is correct

`nn.Conv2d(3, 16, 5, padding=2)` — the first argument (3) is `in_channels` (matching the input image's 3 RGB channels), and the second argument (16) is `out_channels`, i.e. the number of learned filters, which becomes the output's channel count: 16. (B: 3 is the *input* channel count, before this layer runs. C and D are the channel counts after the second and third conv layers respectively, not the first.)

How many trainable parameters are there in the **second conv layer**? (including biases)

A. 1,536

B. 4,640

✓ CORRECT

C. 9,216

D. 32

Correct answer: B

Full worked solution

Second conv layer: `nn.Conv2d(16, 32, 3, padding=1)` — `in_channels=16`, `out_channels=32`, `kernel=3×3`.

$$\text{Weights} = (3 \times 3 \times 16) \times 32 = 144 \times 32 = 4608$$

$$\text{Biases} = 32 \text{ (one per output filter)}$$

$$\text{Total} = 4608 + 32 = \mathbf{4640}$$

Where the wrong options likely come from: A (1,536) would be $3 \times 3 \times 16 \times 32$ without... actually $144 \times 32 = 4608$, so 1,536 looks like it comes from omitting a factor (e.g. $3 \times 16 \times 32 = 1536$, forgetting one of the two kernel dimensions). C (9,216) is exactly the parameter count of the *third* conv layer's weights alone ($3 \times 3 \times 32 \times 32 = 9216$ — close to, but not exactly, this layer's numbers, a plausible mix-up target). D (32) is just the bias count alone, forgetting the weights entirely.

How many trainable parameters are there in the **third conv layer**? (including biases)

A. 1,792

B. 5,824

C. 18,496

✓ CORRECT

D. 64

Correct answer: C

Full worked solution

Third conv layer: `nn.Conv2d(32, 64, 3, padding=1)` — `in_channels=32`, `out_channels=64`, `kernel=3×3`.

$$\text{Weights} = (3 \times 3 \times 32) \times 64 = 288 \times 64 = 18432$$

$$\text{Biases} = 64 \text{ (one per output filter)}$$

$$\text{Total} = 18432 + 64 = \mathbf{18496}$$

(D — 64 — is just the bias count alone; A and B don't correspond to any straightforward correct sub-computation for this layer, and are there purely as plausible-looking distractors.)